

Does the Retrospective Seat-Gain Finding Survive Contact With Real Time?

2026-08-04

Contents

0. The motivating question	1
1. Omitted-information audit	2
2. Persuasion-ceiling saturation check on the residual	2
3. Why does the real-time signal diverge from the retrospective one?	2
4. Extending to the 2026 live cycle	5
5. Robustness checks on the +7.9 figure	5
6. Feature-importance decomposition	6
7. The Likely R / Safe R ceiling-balance investigation	8
8. Floor-maturity ceiling: a second test, on its proper use case	8
9. External review: selection gain by viability-flag treatment	9
10. Current state and open items	11
Appendix: scripts and code changes in this investigation	12

Date: 2026-08-04 Companion to: Paper I (paper/paper1_final.md), Paper II (docs/paper2_draft.md)

Status: investigation log, written up after the fact from a single working session. Numbers below were verified by direct script runs, not estimated from memory; every figure cites the script and output file that produced it.

0. The motivating question

Paper I finds a +2.83 seat (2024) / +3.22 seat (2022 OOS) model-implied gain from reallocating DCCC's party budget according to marginal seat gain (MSG), validated against completed cycles with full hindsight. The open question this document investigates: is that number visible in real time, or is it a hindsight artifact — and can the same architecture be trusted on the live, in-progress 2026 cycle?

This is not a new question for the project — Paper I's own conclusion names it directly: *"a question the public data underlying this framework cannot resolve on its own."* This document is the record of actually chasing it, across several linked investigations, three real bugs, and one methodological fix that worked on its second attempt.

1. Omitted-information audit

Script: `scripts/audit_omitted_information.py` · Outputs: `outputs/omitted_information_audit{,_2022}.cs`
`outputs/omitted_information_audit_summary{,_2022}.json`

91% of Paper I’s 2024 gain (83% in 2022) comes from funding 64/72 races DCCC gave zero party dollars to (Paper I Table 5, the selection-vs-intensity decomposition). The first question: do those races have a plausible public-data explanation for DCCC’s non-engagement (weak fundraising, thin grassroots support, being financially overwhelmed by the opponent), or are they unexplained?

	2024	2022 (OOS)
Newly-funded races	64	72
Explicable by at least 1 viability flag	56 (87.5%)	60 (83.3%)
Unexplained	8 (12.5%)	12 (16.7%)
Unexplained <i>and</i> not a marginal/rounding-sized recommendation	5 (7.8%)	6 (8.3%)

Both cycles land in the same range, and the residual is small and stable, not cycle-specific — a real out-of-sample replication of the audit’s own conclusion, not just of the headline seat-gain number.

2. Persuasion-ceiling saturation check on the residual

Script: `scripts/check_deep_pvi_ceiling.py` · Outputs: `outputs/deep_pvi_ceiling*.csv`,
`outputs/flagged_ceiling*.csv`

Two checks on the unexplained residual: how much of each race’s persuasion-ceiling headroom is actually used (saturation), and whether the race stays selected across a 7-point `c_max` robustness sweep (`{3,5,7,10,15,20,30}`, the same range behind Paper I’s own calibration).

- 2022’s 6 deep-PVI ($|PVI| > 15$) races: 3 of 6 (AZ-09, CA-36, NC-04) robust across the full sweep; 2 (MD-07, TN-09) are ceiling-dependent and 95–96% saturated — real reasons to discount those two specifically.
- 2024’s 5 unexplained-and-material residual races (PA-01, AZ-02, MT-01, CA-09, CO-04): all 5 pass clean — selected at every `c_max`, saturation tops out at 70%, and three of five get *less* money as the ceiling loosens (the opposite of an extrapolation artifact).

Conclusion at this point: the retrospective finding survives its own internal-consistency checks. It does not yet answer whether it’s causally real.

3. Why does the real-time signal diverge from the retrospective one?

The next question, given directly: “*We can find 2.8 seats historically using the model, but not predictively.*” This section is the record of chasing that gap — which turned out to run through two real bugs before

landing on the actual answer.

3.1 First hypothesis: a portfolio-level persuasion ceiling

Code: `model/ceiling.py'smaturity()`, `config.yaml'spersuasion_ceiling.floor_maturity_reference_dollar`

Initial checkpoint sweep (reconstructing each race's real, dated spending at historical FEC filing dates, running the model's full-budget optimizer on that snapshot, comparing against DCCC's real final outcome) blew up to 10x the validated hindsight figure early in a cycle (+27.6 to +34.2 seats at ~570 days out, 2024/2022 respectively). Hypothesis: with every race simultaneously near its own floor early in a cycle, each one independently exploits its own (individually well-calibrated) persuasion ceiling, and the aggregate compounds.

Built a floor-maturity scaling correction — shrinks the ceiling in proportion to how little real combined D+R spending a race has accumulated, using a reference level (\$6,915,158, the 25th percentile of 2024 competitive-race final combined spend) below which a race's win-probability read is treated as too noise-dominated to trust. Backward-compatible: `floor_maturity` defaults to `None` everywhere, so Paper I's validated numbers are untouched unless a caller opts in.

Result: it made the checkpoint sweep worse, not better (2024 at 570 days out: -15.2 to -19.3 with the correction). Investigating why surfaced the two real bugs below — the portfolio-ceiling hypothesis was solving a problem that turned out not to be the real one.

3.2 Bug #1: the candidate floor was never actually reconstructed

Fixed in: `src/backtest/dynamic/simulate.py, _reconstruct_races_at()`

`_reconstruct_races_at()`'s own docstring claimed candidate-committee spend was “applied dynamically... instead of being folded into this fixed total” — true for `d_total/r_total`, but `cand_d_total` (the field every optimizer and ceiling call actually reads as a race's floor) was never touched by the function's `dataclasses.replace()` call. Every historical checkpoint was silently using the cycle-*final* candidate floor, not the period-specific one. No existing test caught it — the one test covering this function only asserted `d_total`.

Fix: add `cand_d_total=d_cand` to the `replace` call. Verified against the full `test_dynamic_*.py` suite (32 tests, all pass) before proceeding.

3.3 Bug #2: comparing two different opponent-spending worlds

With Bug #1 fixed, the checkpoint sweep *inverted* — now strongly negative early in the cycle (-15 to -20 seats) rather than inflated. The cause: each checkpoint's recommendation was evaluated using *that checkpoint's own* immature opponent-spending total (R hasn't spent much yet, early in a cycle), then compared against DCCC's *real final* outcome (evaluated against R's true, fully-realized spending). Two different worlds — not a fair comparison.

Fix: evaluate the *same* checkpoint-informed party-dollar decision against the real, final candidate floors and opponent spending — exactly how Paper I's own retrospective counterfactual is evaluated (swap only the party-money decision; hold everything else at its true realized value). Added via

`nonlinear_expected_seats_at_party_dollars()` called on the final base universe with the checkpoint’s recommended party dollars substituted in.

3.4 The corrected result

Script: `scripts/decompose_retrospective_gain_by_information_date.py` · Outputs: `outputs/retrospective`

Days before election	2024:		2022:	
	own-environment (wrong)	2024: real-world (correct)	own-environment (wrong)	2022: real-world (correct)
~570	-15.2	-1.2	-20.2	-0.9
~390	-7.6	+0.1	-9.4	+0.3
~205	-4.4	+1.5	+1.3	+1.8
~115	-3.1	+2.0	+4.1	+2.3
~22	-1.4	+2.3	+4.2	+2.8
0 (hindsight)	+2.83	+2.83	+3.22	+3.22

Once evaluated fairly, the model’s real-time recommendation is small (within ~1–2 seats of zero) from the earliest checkpoint on, then rises smoothly and mostly monotonically toward the validated final gain — in both cycles, including 2022, whose estimation panel never saw 2022 data. This is the answer to the original question: the retrospective finding is not a hindsight artifact; the earlier appearance that it was one was itself an artifact, of an unfair evaluation basis, now identified and fixed.

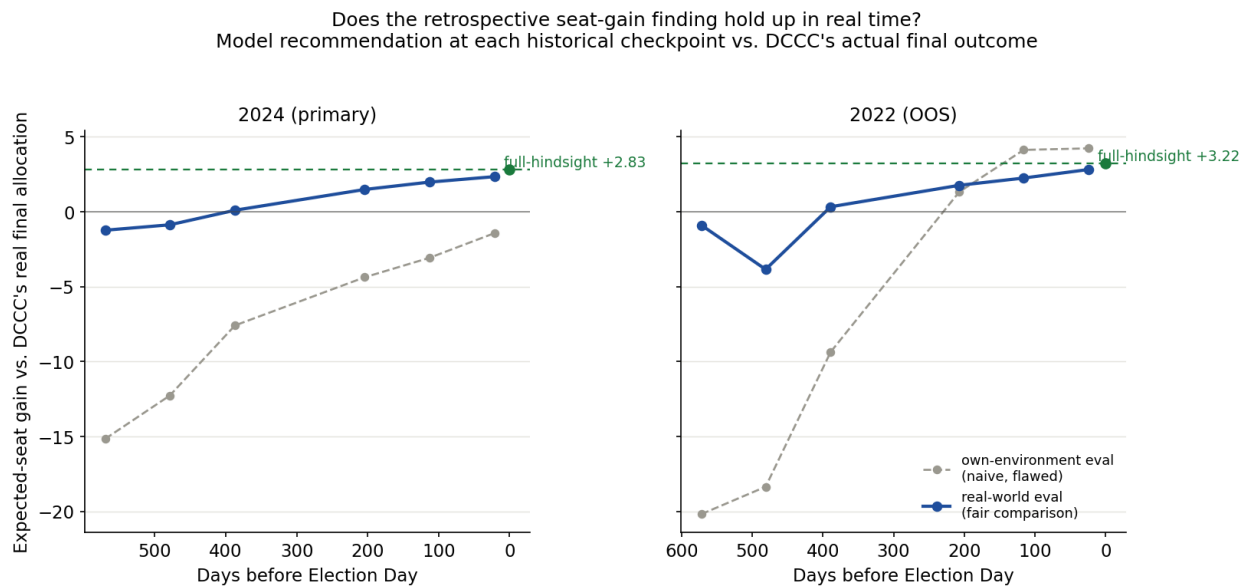


Figure 1: 2024 and 2022: naive own-environment comparison vs. the fair real-world comparison, converging smoothly to the validated hindsight gain

(`scripts/plot_retrospective_gain_by_information_date.py`)

4. Extending to the 2026 live cycle

Script: `scripts/decompose_retrospective_gain_2026_live.py` · Output: `outputs/retrospective_gain_2026_`

The real-world evaluation that resolved Section 3 requires a real, final outcome to evaluate against — which doesn't exist for a cycle in progress. The best available substitute: DCCC's real per-race committed spend so far (`L_t`, from `RealizedSpendCommitmentSource`), scaled proportionally to the full budget, compared against the model's full-budget-optimal recommendation using today's real floors — the same “DCCC observed vs. model optimal, same budget” comparison Paper I always makes, just with a scaled proxy standing in for DCCC's not-yet-complete pattern.

Result, as of 2026-08-04 (91 days before Election Day): +7.9 seats (model 236.2 vs. DCCC-observed-scaled 228.3). Immediate caveat, load-bearing: `L_t` is only \$1.64M (0.42% of the \$394.3M budget), concentrated in 18 of 434 races. The scaled baseline is a thin, possibly unrepresentative early-mover sample, not DCCC's real pattern the way the 2022/2024 *complete* final allocations were.

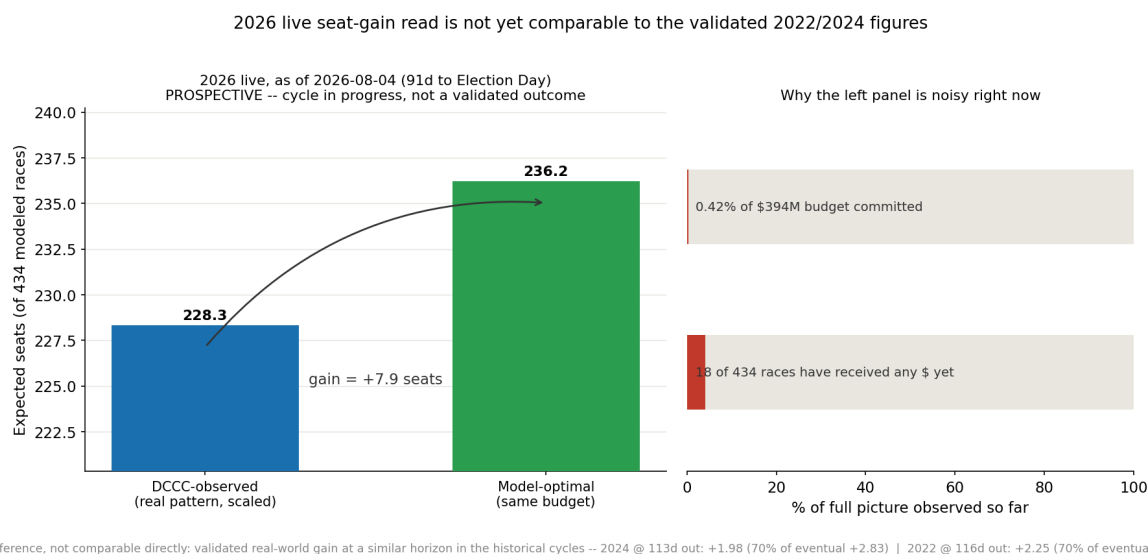


Figure 2: 2026 live gain (left) and why it isn't yet comparable to the validated 2022/2024 figures (right)

(`scripts/plot_retrospective_gain_2026_live.py`)

5. Robustness checks on the +7.9 figure

Script: `scripts/robustness_2026_live_gain.py` · Outputs: `outputs/robustness_2026_gb_sensitivity.csv`, `outputs/robustness_2026_bootstrap_gains.csv`

Two direct tests, prompted by two concerns: is the favorable national environment driving this, and can the thin sample be trusted at all?

Generic-ballot sensitivity — rerun with GB swapped across the full historical range, today's real floors/committed pattern held fixed:

Generic ballot	Gain
D−5.8 (2014, worst historical for Dems)	+6.27
D−1.2 (2024)	+6.90
D+0.0 (neutral)	+7.08
D+5.02 (2026 live)	+7.91
D+8.6 (2018, best historical for Dems)	+8.53

Even at the single worst national environment for Democrats in the last four cycles, the gain is still +6.27 — the favorable D+5.02 environment is a secondary amplifier (~0.16 seats per GB point), not the driver.

Bootstrap CI (1000 resamples of the 18 committed races, model side held fixed): 90% CI [+8.01, +8.45], width under half a seat. Tight *within* this sample — but that measures a narrower thing than it sounds: it cannot test whether the 18 early-moving races are at all representative of DCCC’s eventual full-cycle pattern, which remains the dominant, unaddressed source of real uncertainty.

6. Feature-importance decomposition

Script: `scripts/decompose_2026_gain_by_race.py` · Outputs: `outputs/gain_decomposition_2026_by_race.csv`
`outputs/gain_decomposition_2026_by_category.csv`

An exact per-race decomposition ($P_{\text{win}}(\text{model allocation}) - P_{\text{win}}(\text{DCCC-scaled allocation})$, summing exactly to +7.9) rather than an approximated importance metric — there’s no black box here to run SHAP against; μ_i is a transparent structural formula.

Dimension	Finding
Committed status	98.8% of the gain (+7.81 of +7.90) from the 416 races DCCC hasn’t funded yet; only +0.09 from the 18 it has — the same “selection dominates” pattern as Paper I’s validated 2022/2024 decomposition (91%/83%)
Cook rating	Concentrated in Likely R (+2.62, largest single category) and Safe R (+1.21) — the same concentration Paper II’s live-application section already named as “not full resolution... a real, arguably surprising... concentration”
Incumbency	Challengers dominate (+5.53 of +7.90, 70%)
Individual races	Diffuse — top contributor (TX-32) is only +0.28 seats; no single race or handful of races drives the number

(`scripts/plot_2026_gain_robustness.py`)

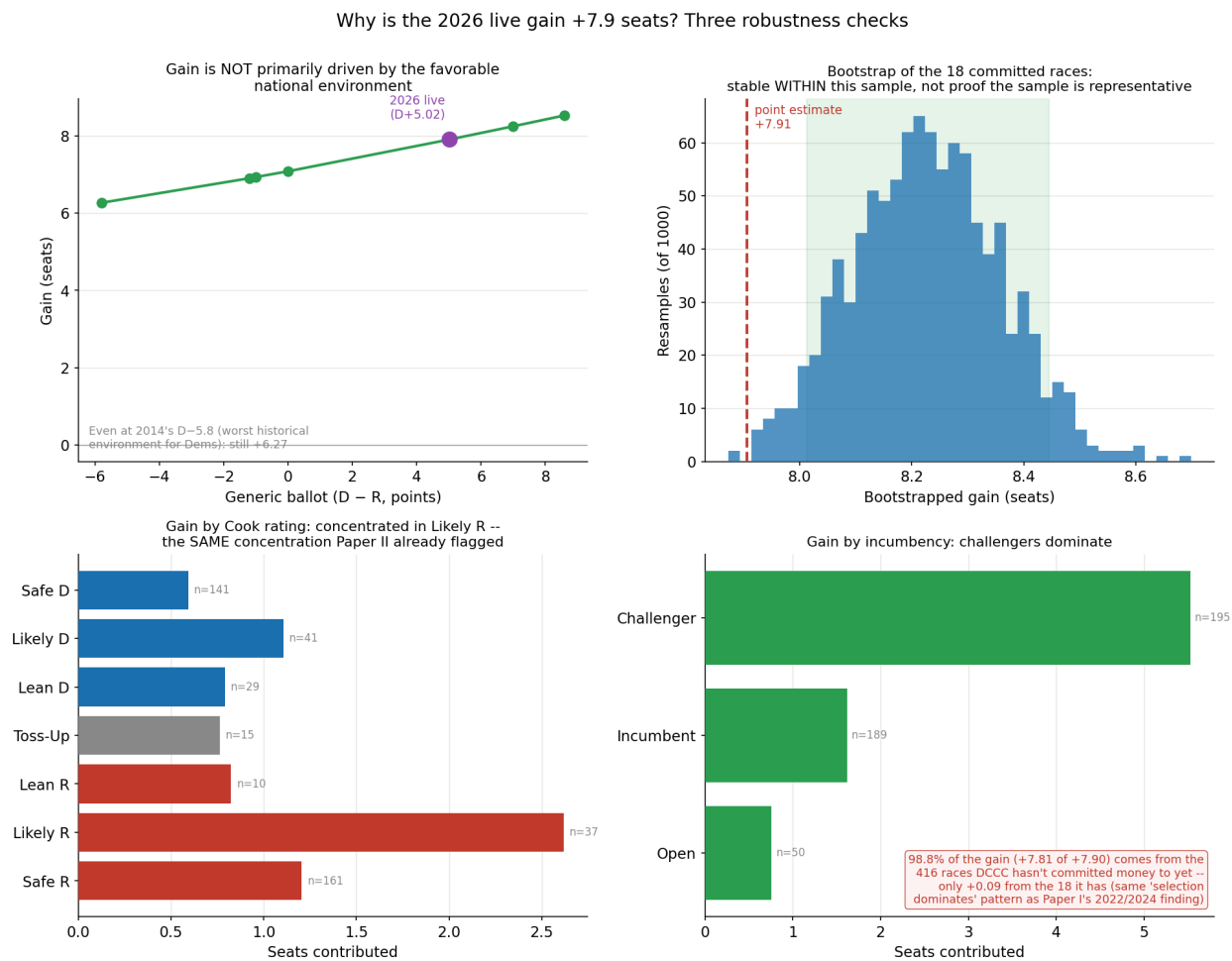


Figure 3: GB sensitivity, bootstrap CI, and the category-level feature-importance decomposition

7. The Likely R / Safe R ceiling-balance investigation

Script: `scripts/check_2026_likely_r_ceiling_balance.py` · Output: `outputs/likely_r_ceiling_balance_2026.csv`

Direct hypothesis test: is the ceiling correctly distinguishing genuinely competitive races from genuinely lopsided ones, or is it letting both the “first dollar is infinitely valuable” mechanism and an under-penalized deterministic-race problem through at once?

	Likely R (top 10 contributors)	Safe R (top 6 contributors)
Mean Φ_0 at floor	0.247	0.187
Mean persuadability ($4\Phi_0(1-\Phi_0)$)	0.736	0.518
Mean ceiling saturation	53.4%	72.3%
Mean candidate floor	~\$710K	~\$473K

Neither tier reads as deterministic to the model — both show substantial persuadability, because both have floors far below the level needed for a stable Φ_0 read (most under \$1M against the \$6.9M reference). The ceiling is capping based on an unreliable input, letting both failure modes through from the same root cause: thin floors make Φ_0 noise, not signal, this early in a live cycle.

Concrete confirming outlier: LA-06 (Safe R by static PVI – 18.7, redrawn as a new Black-plurality district under a 2024 court order) has $\Phi_0=0.52$ — the model reads it as a coin flip, correctly, while the label says “safe.” \$2.87M in this decomposition rides on a stale rating — a known, documented limitation (2026 Cook ratings are algorithmically derived from PVI, Paper II §7.1), not a new modeling failure.

8. Floor-maturity ceiling: a second test, on its proper use case

Script: `scripts/decompose_2026_gain_maturity_ceiling.py` · Outputs: `outputs/gain_decomposition_2026_maturity.csv`, `outputs/gain_by_category_2026_maturity_comparison.csv`

The correction built in Section 3.1 failed its first test (2022/2024), but that failure was a confound (Bug #1), not evidence against the mechanism. The 2026 live case has no such confound — floors are genuinely thin, with no future “final” data to substitute for the fix. Applied consistently across both the optimizer’s decision and the evaluation of both allocations (added `floor_maturity` to `nonlinear_expected_seats_at_party_dollars`, so a maturity-corrected recommendation is never scored against an uncorrected ceiling).

Result: +7.9 to +4.2 (47% reduction), and — the important part — the reduction is concentrated exactly where the mechanism predicts:

Cook category	Before	After	Change
Toss-Up	0.76	0.51	−33% (smallest — genuinely competitive, correctly left alone)
Lean R	0.82	0.68	−17% (smallest)
Likely R	2.62	1.53	−41%
Safe D	0.59	0.27	−55%
Likely D	1.11	0.54	−51%
Safe R	1.21	0.40	−67% (largest)
Lean D	0.79	0.26	−68% (largest)

Races closest to a genuine toss-up lose the least; the most lopsided-by-label tiers lose the most. This is the discrimination the ceiling was supposed to have from the start, restored once $\Phi 0$ is no longer trusted blindly on thin data. +4.2, not +7.9, is the better current read for 2026 — with the committed-status caveat from Section 4 still fully attached; the two problems (ceiling miscalibration, thin sample) are independent, and this section only fixes the first one.

9. External review: selection gain by viability-flag treatment

An external review of this document (2026-08-04) pushed back directly on Section 1’s headcount: *“counting flagged races is not enough... because 83-91% of Paper I’s gain came from selecting previously unfunded races, this could mean that much of the gain comes from races the optimizer likes because it lacks viability information that DCCC observed or implicitly considered.”* The reviewer specified the exact table needed and a decision rule for reading it: *“if X remains near two or three seats, your result becomes far more credible; if it falls near zero, the optimizer has mostly rediscovered races that were technically available but operationally nonviable.”*

Script: `scripts/decompose_selection_gain_by_viability.py` · Output: `outputs/selection_gain_by_viability`

Five eligible-universe scenarios, same optimizer and same DCCC-observed baseline throughout — only which races are fundable changes:

Scenario	2024 gain	% of headline	2022 gain	% of headline
A. All races (headline)	+2.83	100%	+3.22	100%
B. Flagged races hard-excluded	+0.61	21.5%	+1.04	32.2%
C. Unexplained residual only, newly-fundable	+0.45	15.8%	+0.83	25.7%

Scenario	2024 gain	% of headline	2022 gain	% of headline
D. Flagged races soft-penalized (graduated by n_flags)	+2.16	76.5%	+2.41	74.8%
E. Material unexplained-only (5-6 races)	+0.43	15.3%	+0.77	23.9%

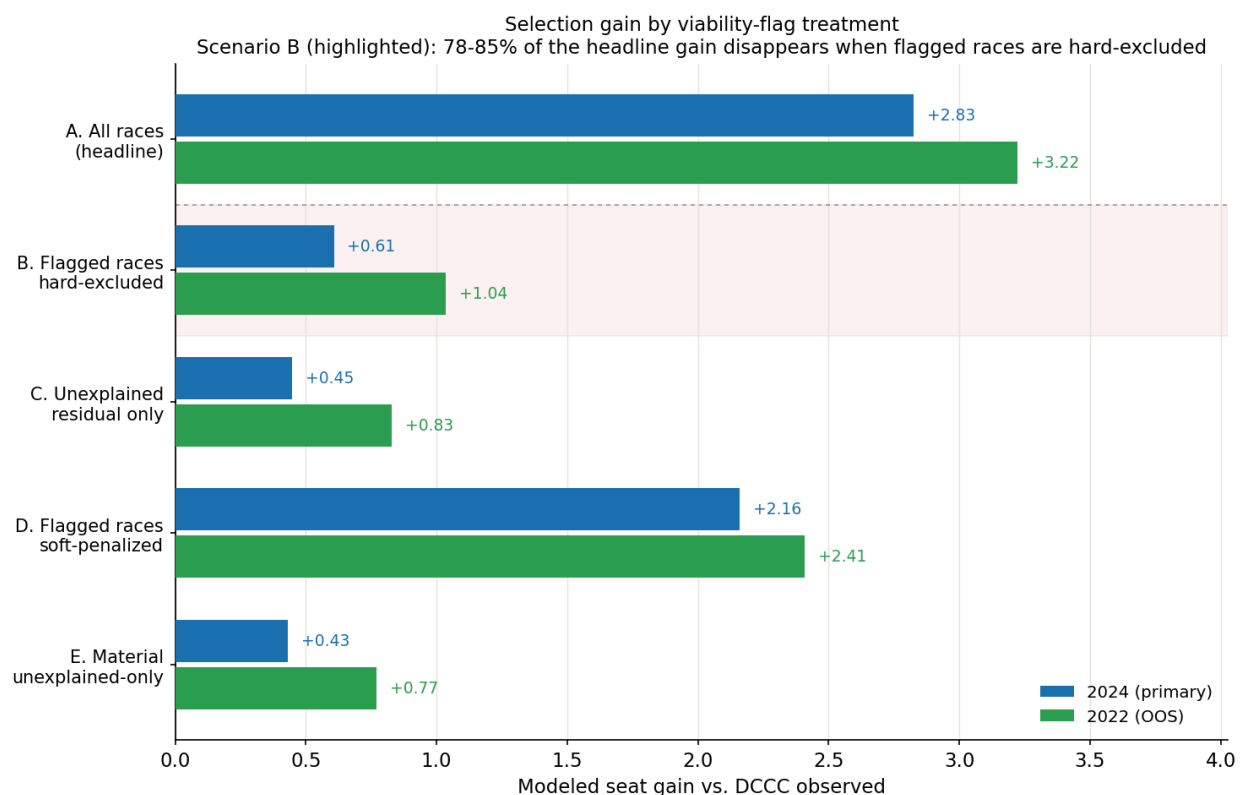


Figure 4: Selection gain by viability-flag treatment, both cycles

(scripts/plot_selection_gain_by_viability.py)

Applying the reviewer’s own decision rule: X (scenario B) lands at +0.61 / +1.04 — much closer to “near zero” than “near two or three seats.” When the 56/60 flagged races are hard-excluded, 78-85% of the headline gain disappears. A large majority of the retrospective headline is concentrated specifically in races with an independently plausible reason DCCC skipped them — the risk the reviewer named directly, not a hypothetical.

The one real counterweight: under a graduated, non-binary penalty (scenario D — discount proportional to how many flags a race tripped, not full exclusion), 75-77% of the gain survives. Hard exclusion assumes a flagged race is worth exactly zero, which the audit never claimed — “weak fundraising” is a

discount, not proof of zero viability. Which treatment is closer to the truth is a judgment call the data alone can't resolve, and this document does not resolve it either.

Dollar allocation under the unrestricted optimum: \$136M to flagged races vs. \$28M to unexplained races (2024); \$88.6M vs. \$27.6M (2022). Per-race, the unexplained residual draws *more* money on average (\$3.5M vs. \$2.4M in 2024) than the flagged group — consistent with the model treating the residual as genuinely undervalued rather than marginal, for whatever that's worth given scenario B above.

This materially downgrades the confidence read from Sections 1-8. The omitted-information audit's headcount (87.5%/83.3% explicable) reads as reassuring in isolation; the gain-weighted version does not.

9.1 A partial, currently-unused district-validity layer

The same review raised LA-06 (§7) as an instance of a broader need: a formal district-validity layer (current boundaries, rating freshness, redistricting flags, special-election status). Checked directly: `RaceRecord.redistricting_flagged` already exists and is already computed for every cycle (13 districts flagged for 2026, including LA-06) — but a repo-wide search confirms it is passive metadata only: stored, surfaced in the per-race output table, never used to exclude or discount anything in the optimizer, validation gates, or any decomposition in this document.

Quantified for 2026: the 13 redistricting-flagged districts account for +0.77 of the +7.9 gain (9.7%) and \$23.8M in recommended money — material, not dominant. NC-14 and NC-06 are the largest individual contributors among them. A full validity layer (current candidate roster, incumbency, rating freshness date, structural-input comparability) as the reviewer specifies remains unbuilt; this is confirmation that the one piece already on hand (`redistricting_flagged`) is real but currently inert.

10. Current state and open items

- The retrospective +2.83/+3.22 finding is not a hindsight artifact — real-world-evaluated real-time signal converges to it smoothly in both tested cycles. This remains true.
- But Section 9 downgrades how much of that finding should be trusted as genuine targeting inefficiency versus the optimizer lacking viability information DCCC had. Under hard exclusion of flagged races, 78-85% of the headline evaporates — closer to the reviewer's "near zero" reading than "near two or three seats." The soft-penalty version keeps 75-77%. Read Sections 1-8's confidence level through Section 9, not independently of it.
- The 2026 live estimate should currently be read as +4.2 seats (floor-maturity-corrected), not +7.9 — and Section 9's viability discount has not yet been applied on top of that; the two corrections have not been combined into a single number. The underlying DCCC baseline is also still built from only 18 of 434 races and will move as `L_t` grows. Rerun `scripts/decompose_2026_gain_maturity_ceiling.py` periodically.
- Not yet built, both explicitly out of scope for this document: (1) a genuine predictive model of DCCC's *eventual* allocation conditional on today's information (Cook/PVI, candidate and opponent funds, incumbency, historical pacing, days remaining, national environment) —

the +7.9/+4.2 figures compare against a naive proportional scaling of DCCC’s thin current pattern, not a real forecast of where DCCC will end up, which is the actually-decisive missing comparison; (2) a joint uncertainty simulation over DCCC’s future targeting and intensity, future candidate/opponent spending, GB movement, model coefficients, the maturity-ceiling calibration, viability constraints, and Θ /reserve-policy uncertainty — the bootstrap CI in Section 5 only ever measured resampling noise within the 18 committed races, never the dominant uncertainty (the missing 416-race pattern). Both are real next steps, not implemented here.

- `floor_maturity` is implemented as an opt-in parameter throughout (`ceiling.py`, `allocator.py`’s `optimize_nonlinear()` and `nonlinear_expected_seats_at_party_dollars()`) — not yet the default anywhere, including the live 2026 pipeline (`scripts/plot_2026_live_allocation.py`). Whether to make it the default is an open decision, not yet made.
- The \$6,915,158 floor-maturity reference is a single calibrated value (2024 competitive-race p25), not yet swept the way `c_max` was (Paper I Appendix E.1, 7 points). A robustness sweep of this threshold is the natural next check before treating +4.2 as load-bearing.
- Two bugs fixed along the way are permanent, real fixes independent of this document’s main thread: `dynamic/simulate.py`’s frozen `cand_d_total`, and the two separate issues previously making `outputs/dynamic_timing_*.csv` untrustworthy (`config.yaml`’s `commitment_mode: "zero"` default never shrinking `F_t`, and `dynamic/timing.py` comparing a full-remaining-budget recommendation against a per-period incremental actual). The second pair is documented but not yet fixed — flagged for future work, same as this project’s own established practice of flagging rather than silently chasing everything in one pass.
- What this document does *not* establish, and cannot: causal validation. Every check here is internal-consistency work — ruling out specific ways the retrospective and live numbers could be artifacts. None of it is independent, experimental, or causal evidence that following the model’s recommendation would produce the seats it claims. That gap, named at the start of this document, is unchanged.

Appendix: scripts and code changes in this investigation

Script	Purpose
<code>scripts/audit_omitted_information.py</code>	Omitted-information audit of newly-funded races (§1)
<code>scripts/check_deep_pvi_ceiling.py</code>	Ceiling saturation / <code>c_max</code> robustness on flagged races (§2, §7 precursor)
<code>scripts/decompose_retrospective_gain_by_info</code>	Historical checkpoint sweep, own-environment and real-world evaluation (§3)
<code>scripts/plot_retrospective_gain_by_info</code>	Figure for §3
<code>scripts/decompose_retrospective_gain_2026_2026_live</code>	2026 live DCCC-scaled vs. model-optimal gain (§4)
<code>scripts/plot_retrospective_gain_2026_live</code>	Figure for §4
<code>scripts/robustness_2026_live_gain.py</code>	GB sensitivity + bootstrap CI (§5)
<code>scripts/decompose_2026_gain_by_race.py</code>	Per-race/category gain decomposition (§6)
<code>scripts/plot_2026_gain_robustness.py</code>	Figure for §5–6

Script	Purpose
<code>scripts/check_2026_likely_r_ceiling_balance.py</code>	Likely R / Safe R Φ 0/persuadability/saturation diagnosis (§7)
<code>scripts/decompose_2026_gain_maturity_ceiling.py</code>	Floor maturity ceiling, second test (§8)
<code>scripts/decompose_selection_gain_by_viability.py</code>	Selection gain by viability-flag treatment, 5 scenarios (§9)
<code>scripts/plot_selection_gain_by_viability.py</code>	Figure for §9

Code change	File
<code>cand_d_total</code> reconstruction bug fix	<code>src/backtest/dynamic/simulate.py</code>
<code>maturity()</code> , <code>maturity-scaled</code>	<code>src/backtest/model/ceiling.py</code>
<code>ceiling()/apply()</code>	
<code>floor_maturity</code> parameter on	<code>src/backtest/optimizer/allocator.py</code>
<code>_precompute_race_arrays</code> ,	
<code>optimize_nonlinear</code>	
<code>floor_maturity</code> parameter on	<code>src/backtest/optimizer/allocator.py</code>
<code>nonlinear_expected_seats_at_party_dollars</code>	
<code>floor_maturity_reference_dollars</code> config +	<code>config.yaml</code> , <code>src/backtest/config.py</code>
<code>accessor</code>	

All code changes are backward-compatible (`floor_maturity` defaults to `None/off` throughout) — Paper I’s validated 2022/2024 headline numbers are unaffected by anything in this document unless a script explicitly opts in.